Curso Superior Tecnológico em

Sistemas para Internet

Java Collections – Map – Implementação HashMap

Instituto Federal do Tocantins

Prof. Helder – helder@ifto.edu.br

HashMap - Definição

HashMap é a implementação da interface Map mais utilizada no Java. Ele armazena dados no formato chave-valor e destaca por oferecer desempenho extremamente alto nas operações básicas, como busca, inserção e remoção de dados no mapa.

Síntaxe do Map

Map<k, v> mapa = new HashMap<>().

Map – estrutura de dados a ser implementada

Mapa – nome dado a variável que armazena o endereço do mapa na memória

k(chave) – tipo objeto que representa a chave associado ao valor, se comparado a um vetor isso seria o índice. Poder ser qualquer tipo de objeto.

v(valor) – tipo objeto é o valor associado a chave. Só é possível ter acesso a esse valor se tiver acesso à chave

HashMap– classe que implmenta a interface Map

Principais Características do HashMap

As principais classes que implementam a interface Map são:

⇒ **Desempenho de alta velocidade:** As operações de busca(get), inserção(put) e remoção(remove) executam em tempo constante;

⇒ **Se ordem garantida:** o HashMap não garante nenhuma ordem para suas chaves ou valroes. A ordem em que o usuário insere não é necessariamente a ordem em que eles serão iterados;

⇒ **Chaves e valores null:** Permite uma única chave null e múltiplos valores null;

⇒ **Duplicatas:** não permite chaves duplicadas

⇒ **Não thread-safe:** por padrão não tem mecanismos para trabalhar com múltiplas threads

Implementação do HashMap

Internamente a forma de armazenado do Map é igual ao HashSet(Conjunto), pois ambos utilizam uma estrutura baseada em tabela hash (hash table) para armazenar e localizar elementos.

A diferença é que o HashMap armazena pares de chave e valor, enquanto o HashSet armazena elementos únicos.

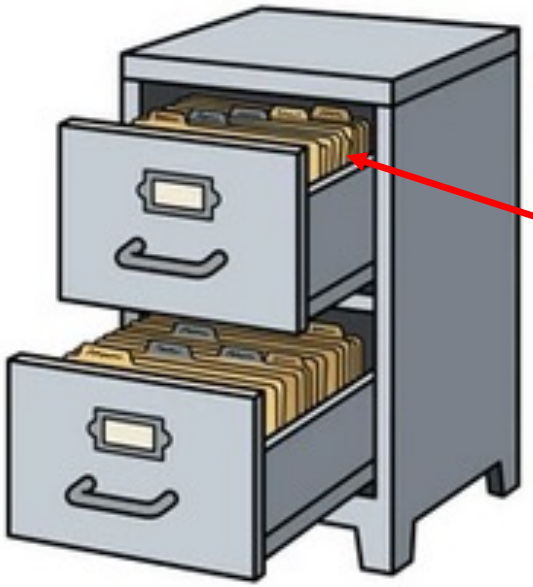
HashMap

Pense o HashMap como uma gaveta de arquivos, onde se navega na vertical e depois na horizontal

Na horizontal pode haver null, um ou vários objetos.

A todo objeto a ser inserido no conjunto é gerado um hash da chavea partir de um método da propria classe HashMap.

HashMap



Objetos: chave-valor

HashMap - Gavetas

Ao se criar um HashMap previamente são criadas 16 gavetas (0-15), ou seja, é criado um vetor de 16 posições onde estará armazenamdo o endereço do primeiro objeto daquela gaveta ou null caso não exista objetos na gaveta. Na prática é armazenado um objeto HashMap que o seu conteúdo será o objeto trabalhado e o endereço do próximo objeto.

Para saber em qual gaveta o objeto será armazenado, gera-se o hash da chave e divide por 16. É o resto dessa divisão que determina em que gaveta o objeto será armazenado.

Os objetos da gaveta são armazenados usando a técnica de Árvore Rubro-Negra

HashMap - Gavetas

Quando objetos são atribuídos a gavetas que já existem objetos diz que houve uma colisão e o objeto será inserido na gaveta e mapeado conforme a técnica Árvore Rubro-Negra.

Se a quantidade de gavetas utilizadas ultrassar 75% a quantidade de gavetas são dobradas e os objetos serão redistribuídos entre as gavetas. A regra para determinação de em qual gaveta o objeto será inserido agora será o resto da divisão entre o hash do objeto e a nova quantidade de gavetas

Princípio Básico

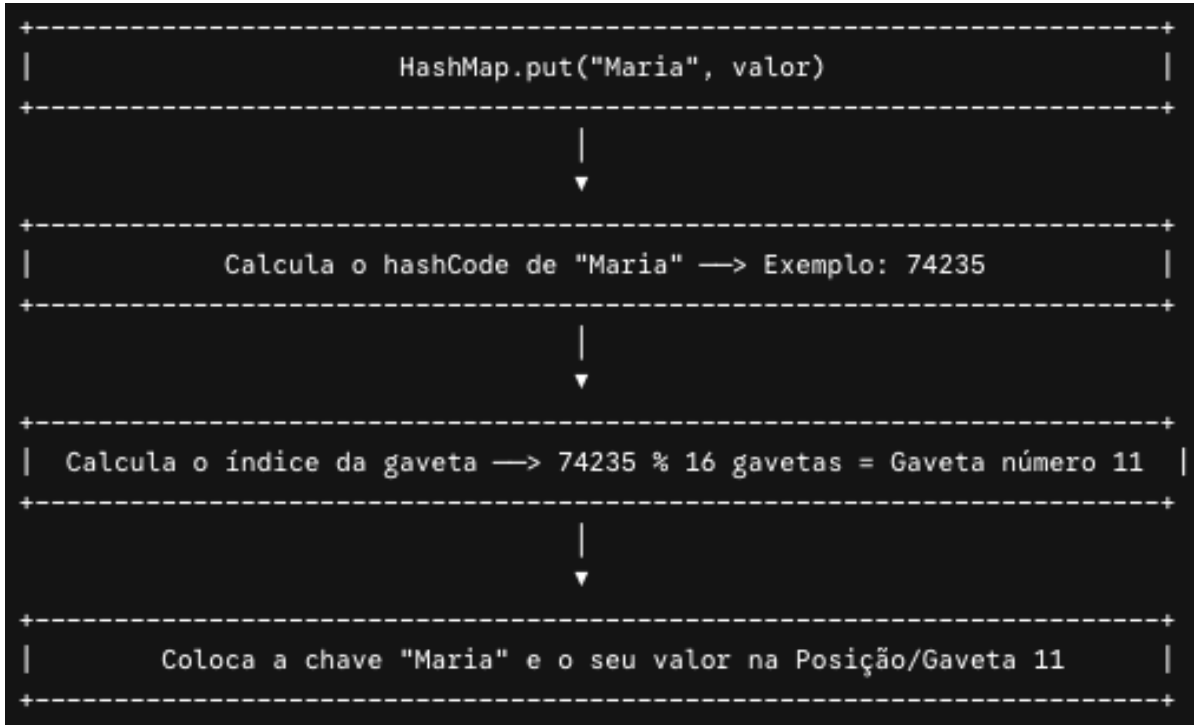
Cada objeto em Java possui um método chamado hashCode(), que calcula um número inteiro único ou representativo da chave associado àquele objeto. Quando um objeto é adicionado ou pesquisado (por exemplo, a chave String "Maria"):

⇒ O Java chama "Maria".hashCode() e recebe um número (ex. 74235)

⇒ O Java pega esse número e faz uma operação matemática simples para descobrir em qual posição do array (gaveta) aquele número se encaixa.

Fórmula simplificada: posicao = hashCode % número_de_gavetas

Princípio Básico



Princípio Básico

Se for adicionar um objeto com chave “Maria” novamente

⇒ O Java calcula o hash, chega a gaveta 11

⇒ Olha o que está lá dentro e compara usando o método .equals()

⇒ Como “Maria”.equals(“Maria”) é verdadeiro, ele percebe que a chave já existe e não adiciona de novo (garantindo a unicidade da chaves) e no caso vai é alterar o objeto associado a essa chave

Princípio da Unicidade das Chaves

Para não permitir chaves duplicadas o HashMap utiliza as técnicas de equals() e hashcode() para comparar os objetos que representa as chaves. Se houver igualdade ele não permite inserir uma nova chave e sim altera os dados associados a essa chave.

Naturalmente se o objeto que representa a chave não houver em sua codificação a implementação dos métodos equals() e hashcode() haverá duplicidade de chaves e comprometimento da estrutura do mapa.

Pesquisa de Valores

Para pesquisar objetos dentro do HashMap é preciso que o objeto tenha em sua implementação o método equal(), caso contrário não é possível encontrar o objeto.

Métodos Específicos - HashMap

Métodos específicos da classe HashMap

=>.getOrDefault(“Maçã”, 0) – retorna o objeto associado à chave Maçã, caso não exista retorna 0

⇒.putIfAbsent(“Maçã”, 50) – adiciona a chave-valor ao mapa, apenas se a chave Maçã não existir no mapa

⇒…

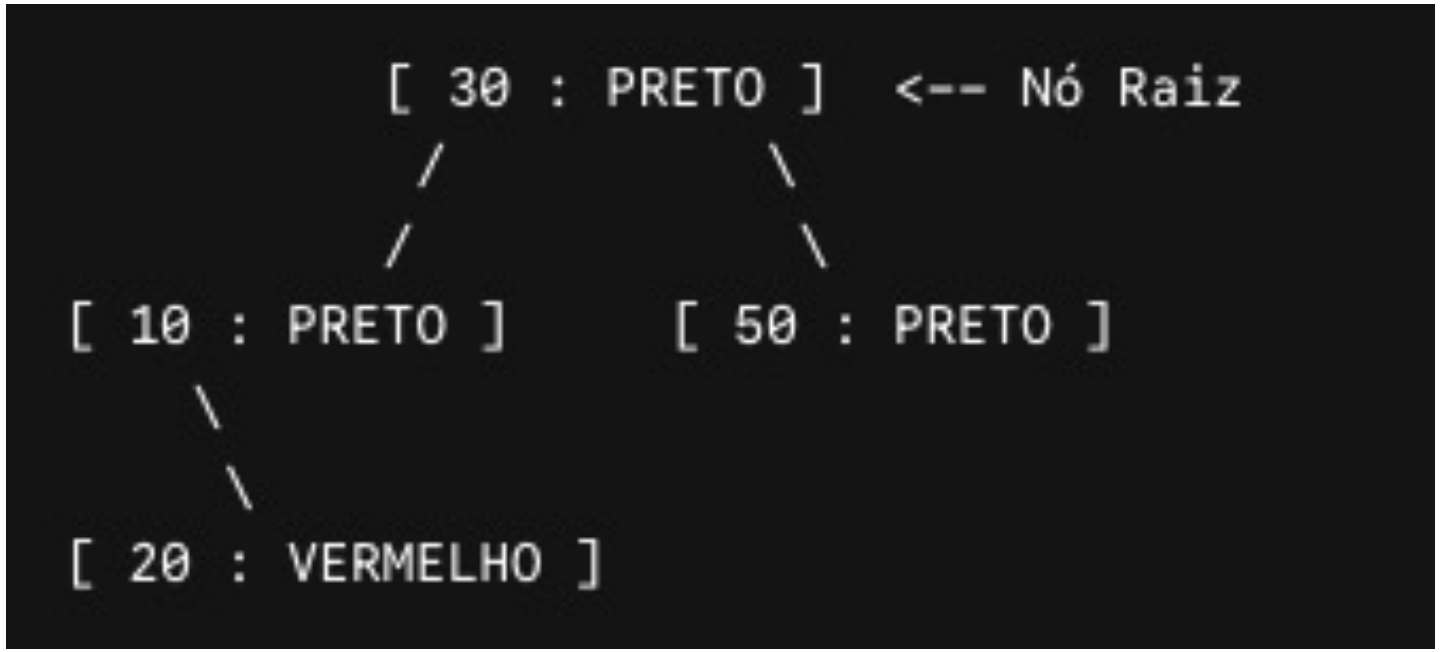
Organização dos Objetos na Memória

O HashMap usa uma árvore chamada Árvore Rubro-Negra (Red-Black Tree) para interligar os objetos do conjunto.

Na memória, essa estrutura é composta por nós soltos(Nodes), onde cada nó guarda o valor do elemento e possui três ponteiros(referências) de memória:

1. Ponteiro para o filho da esquerda(left)
2. Ponteiro para o filho da direita(right)
3. Ponteiro para o nó pai (parente)
4. Uma flag de cor (Vermelho ou Preto), usada para manter a árvore balanceada

Organização dos Objetos na Memória (Conjunto de Inteiros)



Organização dos Objetos na Memória

A Árvore Rubro-Negra(Red-Black Tree) – é um tipo especial de Árvore Binária de Busca (Binary Search Tree ou BST).

Conceito de Bysca em BST:
=>Todo valor menor que o nó atual vai para a esquerda
=> Todo valor maior que o nó atual vai para a direita

Organização dos Objetos na Memória

A Árvore Rubro-Negra é uma árvore auto-balanceada, ela resolve o problema do desgeneramento(gerar apenas uma fila) “pintando” cada nó de vermelhou ou de preto e aplicando 4 regras rígidas.

1. A raiz é sempre preta
2. Nós vermelhos não podem ter filhos vermelhos (não pode haver dois nós vermelhos seguidos)
3. Caminhos de busca iguais: Todo caminho da raiz até as folhas vazias tem que passar exatamente pelo mesmo número de nós pretos
4. Folhas novas tem que ser inserida como vermelhos

Organização dos Objetos na Memória

Quando é adicionado um novo objeto e ele ameaça desequilibrar a estrutura, o Java faz duas operações automáticas:

1. Recolonização: Troca a cor dos nós (de vermelho para preto ou vice versa)
2. Rotação: “Gira” a árvore para rearranjar quem é pai e quem é filho, mantendo a altura equilibrada

Organização dos Objetos na Memória

Simulação de inserir 5 objetos do tipo inteiro, como ficaria a estrutura na memória. Ineria nesta ordem cronológica: 5, 10, 11, 100, 25, 2

Organização dos Objetos na Memória

Elementos a serem inseridos: 5, 10, 11, 100, 25, 2

Inserindo o 5

```
[ 5 : PRETO ]
```

Inserindo o 10
Compara com 5 -> 10 > 5 (vai para a direita). Entra como vermelho

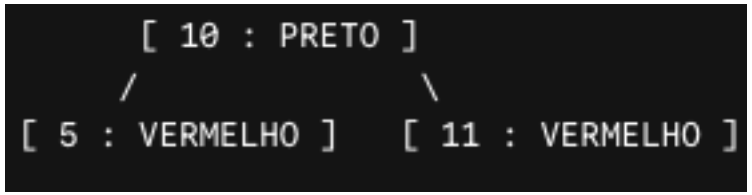
```
[ 5 : PRETO ]
      \
   [ 10 : VERMELHO ]
```

Organização dos Objetos na Memória

Elementos a serem inseridos: 5, 10, 11, 100, 25, 2

Inserindo o 11

Compara com 11 -> 5 (direita); compara com 10 -> 11 (direita).
A árvore fica temporariamente "torta" para a direita com dois nós vermelhos seguidos (10 e 11). O Java realiza uma **rotação para a esquerda** e ajusta as cores para rebalancear

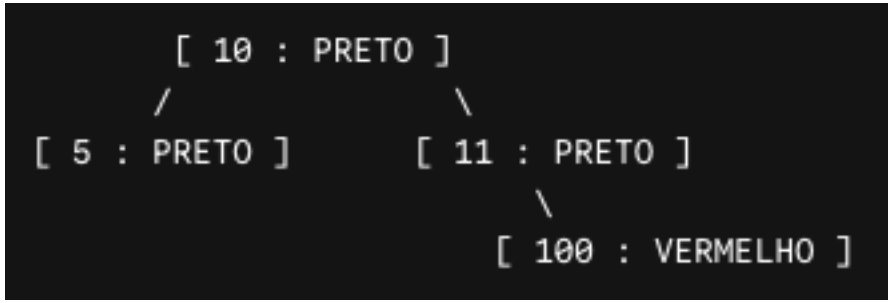


Organização dos Objetos na Memória

Elementos a serem inseridos: 5, 10, 11, 100, 25, 2

Inserindo o 100

Compara com 10 -> 100 (direita); compara com 11 -> 100 (direita).
Entra como filho direto do 11(vermelho). O Java faz uma recolonização nos nós filhos

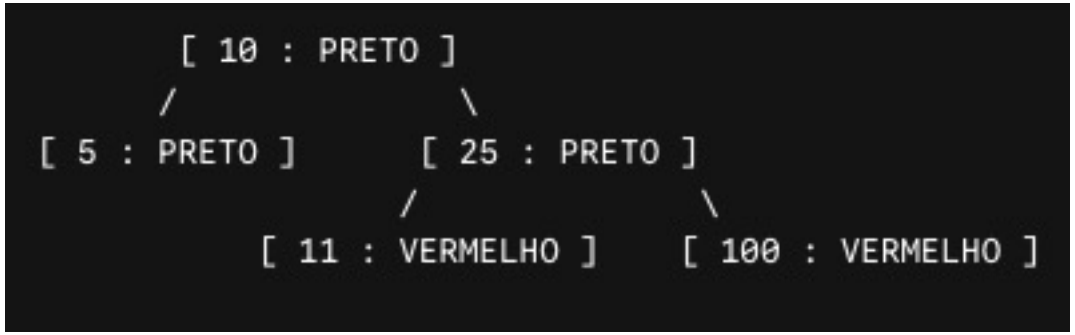


Organização dos Objetos na Memória

Elementos a serem inseridos: 5, 10, 11, 100, 25, 2

Inserindo o 25

Compara com 10 -> 25 (direita); compara com 11 -> 25 (direita); Compara com 100 -> 25 (esquerda)
Dois vermelhos em sequencia(100 e 25). O Java realiza uma rotação para a direita no nó 100 e reajusta as cores



Organização dos Objetos na Memória

Elementos a serem inseridos: 5, 10, 11, 100, 25, 2

Inserindo o 2

Compara com 10 -> 2 (esquerda); compara com 5 -> 2 (esquerda);

